

Signaling Server Setup Guide

Prerequisites

- Node.js 18+ installed
- npm or yarn package manager
- Public IP address or domain name
- (Optional) SSL certificate for HTTPS/WSS

Installation

1. Install Dependencies

```
cd signaling-server
npm install
```

2. Configure Environment

Copy the example environment file:

```
cp .env.example .env
```

Edit `.env` with your configuration:

```
# Server Configuration
PORT=3000
NODE_ENV=production

# Cloudflare Configuration
CLOUDFLARE_ENABLED=true
ALLOWED_ORIGINS=https://yourdomain.com,https://www.yourdomain.com

# Session Configuration
SESSION_TIMEOUT_MS=3600000      # 1 hour
CLEANUP_INTERVAL_MS=60000       # 1 minute

# Logging
LOG_LEVEL=info
```

3. Build the Project

```
npm run build
```

4. Start the Server

Development:

```
npm run dev
```

Production:

```
npm start
```

Deployment Options

Option 1: PM2 (Recommended for Production)

Install PM2

```
npm install -g pm2
```

Start with PM2

```
pm2 start dist/index.js --name cloud-gaming-signaling
pm2 save
pm2 startup
```

Monitor

```
pm2 status
pm2 logs cloud-gaming-signaling
pm2 monit
```

Restart

```
pm2 restart cloud-gaming-signaling
```

Option 2: Systemd Service

Create `/etc/systemd/system/signaling-server.service` :

```
[Unit]
Description=Cloud Gaming Signaling Server
After=network.target

[Service]
Type=simple
User=nodejs
WorkingDirectory=/opt/signaling-server
Environment="NODE_ENV=production"
ExecStart=/usr/bin/node dist/index.js
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Enable and start:

```
sudo systemctl daemon-reload
sudo systemctl enable signaling-server
sudo systemctl start signaling-server
sudo systemctl status signaling-server
```

View logs:

```
sudo journalctl -u signaling-server -f
```

Option 3: Docker

Create `Dockerfile` :

```
FROM node:18-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --only=production

COPY . .
RUN npm run build

EXPOSE 3000

CMD ["node", "dist/index.js"]
```

Build and run:

```
docker build -t cloud-gaming-signaling .
docker run -d -p 3000:3000 --name signaling-server cloud-gaming-signaling
```

Nginx Reverse Proxy

For production deployment with HTTPS:

1. Install Nginx

```
sudo apt-get update
sudo apt-get install nginx
```

2. Configure Nginx

Create `/etc/nginx/sites-available/signaling-server` :

```

upstream signaling_backend {
    server localhost:3000;
    keepalive 64;
}

server {
    listen 80;
    server_name signaling.yourdomain.com;

    # Redirect to HTTPS
    return 301 https://$server_name$request_uri;
}

server {
    listen 443 ssl http2;
    server_name signaling.yourdomain.com;

    # SSL Configuration
    ssl_certificate /etc/letsencrypt/live/signaling.yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/signaling.yourdomain.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    # Logging
    access_log /var/log/nginx/signaling_access.log;
    error_log /var/log/nginx/signaling_error.log;

    # WebSocket Support
    location / {
        proxy_pass http://signaling_backend;
        proxy_http_version 1.1;

        # WebSocket headers
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";

        # Standard proxy headers
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Timeouts
        proxy_connect_timeout 7d;
        proxy_send_timeout 7d;
        proxy_read_timeout 7d;
    }
}

```

3. Enable Site

```

sudo ln -s /etc/nginx/sites-available/signaling-server /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx

```

4. SSL Certificate (Let's Encrypt)

```
sudo apt-get install certbot python3-certbot-nginx
sudo certbot --nginx -d signaling.yourdomain.com
```

Cloudflare Setup

See [CLOUDFLARE_SETUP.md](#) (./CLOUDFLARE_SETUP.md) for detailed Cloudflare configuration.

Firewall Configuration

UFW (Ubuntu)

```
sudo ufw allow 3000/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

iptables

```
sudo iptables -A INPUT -p tcp --dport 3000 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT
sudo iptables-save | sudo tee /etc/iptables/rules.v4
```

Health Checks

Check Server Status

```
curl http://localhost:3000/health
```

Expected response:

```
{
  "status": "healthy",
  "timestamp": "2024-01-01T12:00:00.000Z",
  "uptime": 3600
}
```

Check Statistics

```
curl http://localhost:3000/api/stats
```

Expected response:

```
{
  "activeSessions": 5,
  "connectedClients": 3,
  "totalVMs": 5,
  "uptime": 3600
}
```

Monitoring

Log Files

Logs are stored in:

- logs/error.log - Error logs
- logs/combined.log - All logs

Log Rotation

Install logrotate:

```
sudo apt-get install logrotate
```

Create /etc/logrotate.d/signaling-server :

```
/opt/signaling-server/logs/*.log {
    daily
    rotate 14
    compress
    delaycompress
    notifempty
    missingok
    copytruncate
}
```

Performance Tuning

Node.js Memory Limit

```
NODE_OPTIONS="--max-old-space-size=2048" npm start
```

Increase File Descriptor Limit

Edit /etc/security/limits.conf :

```
* soft nfile 65536
* hard nfile 65536
```

TCP Tuning

Edit /etc/sysctl.conf :

```
net.core.somaxconn = 4096
net.ipv4.tcp_max_syn_backlog = 4096
net.ipv4.ip_local_port_range = 10000 65535
```

Apply:

```
sudo sysctl -p
```

Security Hardening

1. Non-root User

```
sudo useradd -r -s /bin/false nodejs
sudo chown -R nodejs:nodejs /opt/signaling-server
```

2. Environment Variable Security

```
chmod 600 .env
```

3. Rate Limiting (Future)

Install express-rate-limit:

```
npm install express-rate-limit
```

4. Fail2ban

Create `/etc/fail2ban/filter.d/signaling-server.conf`:

```
[Definition]
failregex = Failed connection attempt from <HOST>
ignoreregex =
```

Troubleshooting

See [TROUBLESHOOTING.md](#) (`./TROUBLESHOOTING.md`) for common issues and solutions.

Backup and Restore

Backup

No persistent data to backup (sessions are ephemeral). Backup:

- Configuration files (`.env`)
- SSL certificates
- Nginx configuration

Restore

1. Reinstall dependencies
2. Restore configuration files

3. Restart service

Scaling

Horizontal Scaling

For multiple signaling servers:

1. Set up Redis for session sharing (future feature)
2. Use load balancer with sticky sessions
3. Deploy in multiple regions

Example Load Balancer Config (Nginx)

```
upstream signaling_cluster {  
    ip_hash; # Sticky sessions  
    server 10.0.1.1:3000;  
    server 10.0.1.2:3000;  
    server 10.0.1.3:3000;  
}
```

Next Steps

1. Set up VM Host Service: [SETUP_VM_HOST.md](#) (./SETUP_VM_HOST.md)
2. Deploy Web Client: [SETUP_WEB_CLIENT.md](#) (./SETUP_WEB_CLIENT.md)
3. Configure Cloudflare: [CLOUDFLARE_SETUP.md](#) (./CLOUDFLARE_SETUP.md)